Dropkicks Inc. level editing manual & documentation
v1.1
by Mk-0

Welcome! This document serves as a manual for editing levels and modding the cartridge Dropkicks Inc., as well as a documentation listing most of the data structures used in the game.

The chapters are listed in order of increasing complexity, with the first ones being simpler tutorials for making custom levels and later ones explaining and documenting things for more advanced modding.

Contents

1. Basic level editor usage

1.1. Getting started

To begin editing, download the editor (level_editor.p8) and source code (dropkicks_inc.p8) cartridges. The source code imports data from the editor so they should be in the same folder. (technically, you could skip the source code and directly modify the compressed cart later but i cannot guarantee it would be easy or a good idea)

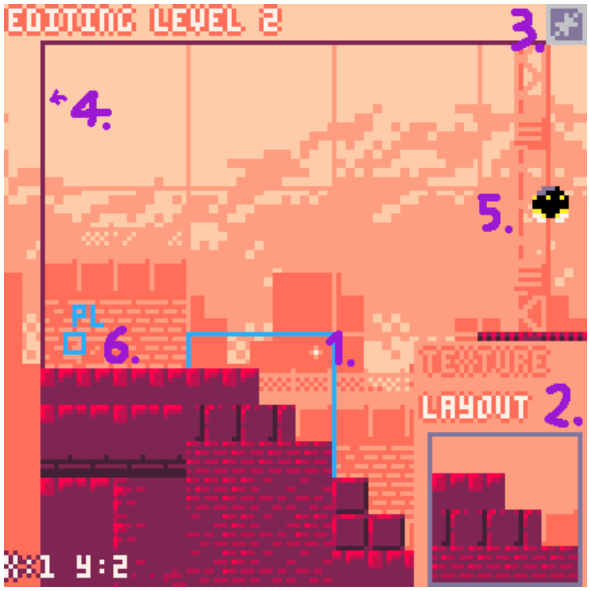
Before starting, it's somewhat important to know how the level data is stored. All levels consist of 3 parts:
- the level header, which is stored as a string line inside a main string array, and contains all the level's settings (like the level name, player starting position, palette, music) and the locations of the other 2 components,
- the level's geometry (all of the terrain & tiles), stored in the map,
- and the entity information (what entities are present in the level and where), which is also stored in the map.

When saving a level, the last 2 things are automatically written onto the map. The header string, however, needs to be manually put into the code. Whenever the editor saves a level, it will put the level's header into the clipboard, which you can then paste into the lvls_info array inside the level editor's code (in tab 6 - data). Select the entire line of the level you're editing by triple clicking, then paste.

1.2. Basic edits

The editor's main menu lists levels alongside a quick preview of the level's backgrounds
Select a level to edit and confirm with 🅾. The manual will use level 2 as an example.

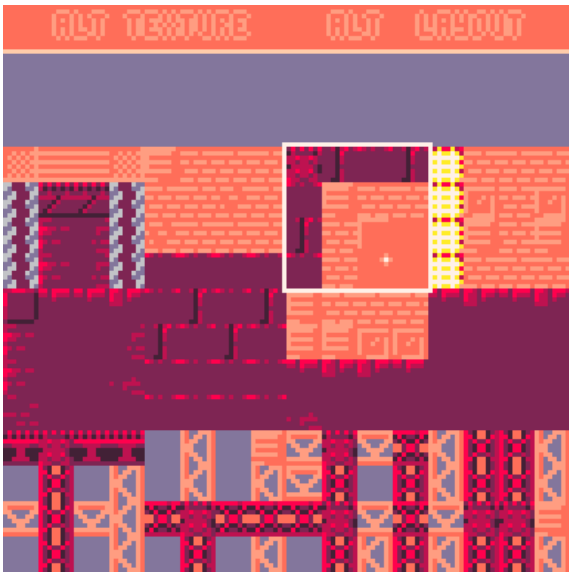
The interface



1. Your cursor (tiny, sorry about that) and the selected tile. Move it with the mouse.
2. Megatile window. You can select megatiles that make up the terrain here.
3. Entity adder/remover. Right now it's grayed out as the level's capacity is filled.
4. Level camera border, usually corresponds with the terrain border.
5. An entity. Can be selected and moved by dragging.
6. The player's spawnpoint. Can also be dragged.

1.2.1. Editing terrain

A level's terrain is organized in a grid of 4x4 tile structures (called megatiles, each corresponding to one tile on the mapspace.)
The bottom right window shows the megatile you have currently selected. Left click to place it onto the grid space your cursor is on.
To pick a new tile, you can either right click on another tile to copy it into the window, or click inside the window to open the tile selection screen. You can also click on the "texture" and "layout" signs to change the megatile's forms.



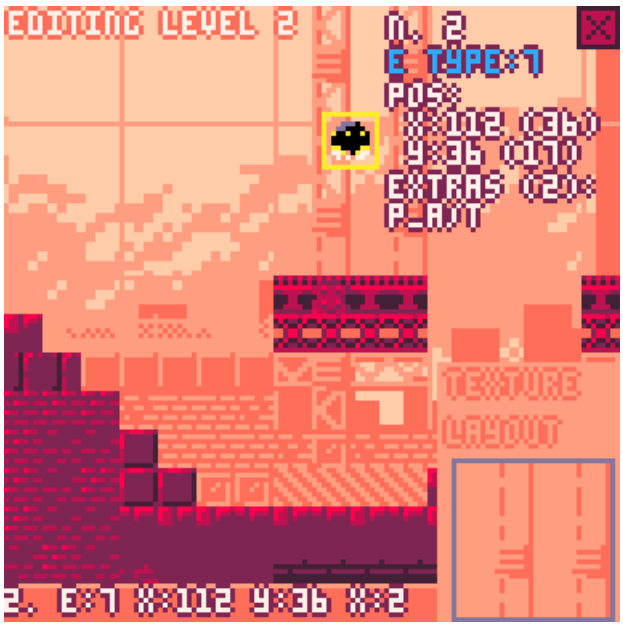
The megatile selector.

Inside the megatile selector, use ⬇ and ⬆ to scroll trough the available megatiles, and left click on one to pick it. All tiles have 4 variants, which you can see by clicking the "alt texture" and "alt layout" buttons on the top.
Alt(ernate) texture changes the tile set the megatiles will use (occasionaly changing their properties), while alt layout changes their, well, layout.

The very first megatile, with the "alt layout" enabled, should be used as the "empty" megatile.

1.2.2. Editing entities

Moving your cursor over the entity will show its hitbox and some stats in the bottom left, and allow it to be edited. Hold left click to move the entity, or press right click to edit its properties.



You can edit 3 properties here - the entity's type, its extra modifications, and its order in the level. Press right click on it again to change which property you're targeting, and ✕ & O to decrease & increase it.
All the current entity types are documented in section 3.3. Entities
The extras modify an entity's behavior, and can be seen in 3.5. Entity extras
The entity's order in the level affects only its position in the map. It's completely unimportant for normal use. (The only time when it could be useful is if you're sharing entity memory between levels, which is too cursed even for me)

Right click anywhere else to stop editing this entity.

The top right button with a "+" sign allows you to add entities. When hovered over, it will also show the level's entity amount and capacity (This is determined by how much map space is to the right of the level's entity memory location).
When a level's capacity is filled, the button is crossed and you cannot add more.
When selecting an entity, this button turns into an "x" delete button. Left click it to delete this entity and clear that memory space.

IMPORTANT: Changing the entity amount affects the level header! Be sure to paste it if you add or remove entities!

1.2.3. Editing settings

Press [PAUSE] and select "level settings" to edit this level's settings. The editor will display a list of all the fields in the level's header.
⬇ & ⬆ to scroll, 🅾 & to decrease/increase the value.
Some fields (like the name) are not numeric and cannot be changed in the editor. You'd need to change them manually in the code.
You can edit the current palette by pressing ➡ or ⬅ on the palette field. ➡ & ⬅ then change which color you're selecting, while ❎ & 🅾 decrease/increase its value.

On the bottom of the list is the background editor (differently colored text), where you can edit the properties of the current two backgrounds. Background properties are saved separately on the map and are not part of the level's header. Many levels share backgrounds so editing one will also affect other levels with that same background!

Press [PAUSE] and "back to editor" to finish editing settings. They are still only saved when you select the "save level" option!

1.2.4. Saving and testing the level

Press [PAUSE] and select "save level" to do this. This will write the level's terrain & entity data to the map, as well as all palette & background changes, and copy them to the cart's permanent storage with cstore().

This will also put the level header in your clipboard as mentioned, which you should now paste into the correct place in the header array (triple click the corresponding line, unless this is the very first or last header). If you're using an external editor, be sure to reload the file first so the map section updates!

A level header, selected

To transfer the changes into the main game, use an external editor and copy/paste the __gfx__ and __map__ sections from the level_editor into dropkicks_inc, replacing the ones there. If you're using the source code version, the #include already copies the header (otherwise copy and replace it there too). If you edited sprites, music or sound effects copy those too.

After that you should be able to test the changes in the main game!

1.3. Adding and deleting levels

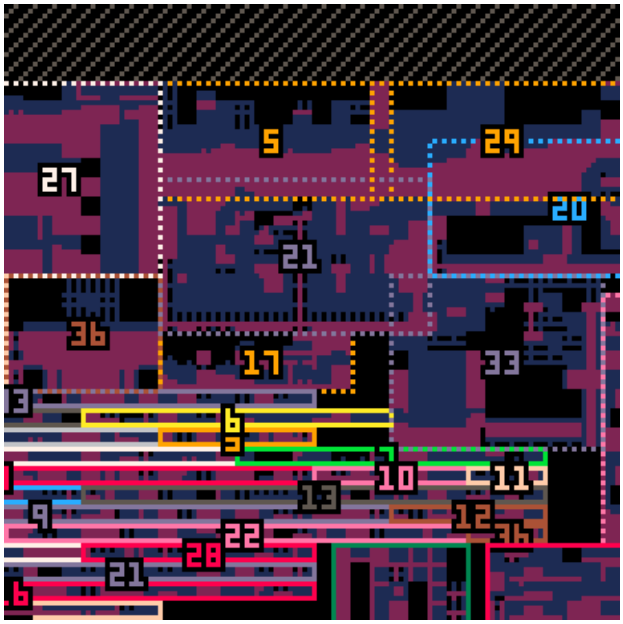
As the game currently uses about 99% of map and compressed space, it's probably a good idea to clear some space before adding new levels.

To delete a level, you could simply remove its line in the main lvls_info array. However, this won't clear the level's map or entity space, so before removing the line, it's a good idea to first go into the level, manually overwrite all the megatiles with air, delete all entities, and then save. This will clear all the used map data - do note that air megatiles are tiles 128, not 0, so empty level space will look not-empty in PICO-8's map viewer.
You can also manually clear the map space after deletion, with the help of the map viewer utility.

To add a level, add a new line into the header array (it's a good idea to copy another level's line then modify it). Then, find some free space in the map and edit the level's map x, y and size properties so it fits there. To be able to add entities to the level, also set the entity count to 0 and set the entity location to a free line in the map.
(Level properties can be checked in section 3.8. Useful entity properties & functions You can also see the formula for converting map space into memory location there.)

1.3.1. Map viewer

In the main menu's pause screen, select "view map". This screen displays the mapspace from [0,12] to [127,43], rendering megatiles and highlighting the regions used by the level terrains, entity lists & other data.
➡, ⬅, ⬇ & ⬆ to move, 🅾 to toggle highlighting, to change zoom level.



The messiest part in the mapspace. Notice how some levels are sharing terrain.

The dotted rectangles with numbers in them correspond to a level's terrain. The horizontal rectangles with numbers are that level's entity data, and the large unmarked green, red and orange rectangles in the bottom right are other data storages (the links, guns and background data).

Using this viewer you can see what regions of the map are used by which levels. If a region is outside all rectangles, it's safe to delete it in PICO-8's map editor (finding it may be difficult, but the map viewer can be paused and resumed inbetween map tweaks and it will update accordingly)

1.4. Exporting

Since the cart takes way too much compressed space to be exported normally, you'd need an external tool to compress the code further.

I used the web version of shrinko-8 (https://thisismypassport.github.io/shrinko8/), and pasted the code there.

Since the last page (data) is referenced from the level editor, you'd also need to manually copy and paste that section before transferring the code into the website.

Before exporting, you could also change the name in the cartdata call (tab 0 line 15) so the mod doesn't share the same save slot as the original cart.
The last section of the cart (__meta:title__, visible only in an external editor) also contains the title and author note, which you could also change alongside the label to indicate the cart is a mod.

2. Advanced editing

2.1. Global variables

Upon being loaded, a level can modify some global variables by specifying them in the 5th place in its header, in order to achieve some special effects. These must be written manualyl inside the code, with the format being [key1,key2/val1,val2].
Below is a list of some useful vatiables a level can modify - the names are very short (and possibly confusing) to save on compressed space in the main cart. Mind the letter cases.

lly - low y limit. [default: 0] - specifies the highest point the camera can see. Negative values will move the limit upwards, positive downwards.
lhy - high y limit. [default: level megatile y size * 32] - the lowest point the camera can see, but also affects how low can the player go before having to restart the level
llx - low x limit [default: 0] - the leftmost point the camera can see
lhx - high x limit [default: level megatile x size * 32] - the rightmost point, and how far right the player needs to go in order to complete the level

grav - level gravity [default: 0.217] - entities that define custom gravity are not affected by this

e_rq - enemy requirement [default: 0] - how many enemies need to be defeated in order to unlock the passage

sl_l - sludge level [default: 1024] - specifies the y level of this level's liquid
sl_c - sludge color [default: 6] - any pixel below the sludge level will have its bottom 3 bytes be overwritten to match this one - the top half of the palette will be color 6 in this case, and the bottom half 14
sl_smth - sludge smoothness [default: 0.982] - an entity's velocity will be multiplied by this number every frame when it's below sludge level
sl_vx - sludge velocity x [default: 0] - add this x velocity to an entity inside the sludge every frame (you can make rivers using this)
sl_vy - sludge velocity y [default: -0.2] - adds y velocity. Simulates buoyancy
sl_dmg - sludge damage [default: 0] - entities inside sludge take this damage every frame. If this is 4.6 or larger entities will also be stunned when inside
sl_r - sludge rise [default: 0] - the sludge will rise (negative) or drop (positive) by this much every frame.
sl_spd - sludge fluctuation speed [default: 5] - how slow will the sludge periodically rise and drop. Higher values make it slower, and will also increase the total amplitude. The starting sludge level is treated as the bottommost point, but this can be set to negative to treat it as the top instead.
sl_h - sludge fluctuation height [default: 0.04] - The amplitude of the fluctuations will be multiplied by this. Also works with negatives.
l_t_c - localized time counter [default: 0] - Mainly can be used to offset the fluctuations even more precisely.

2.2. Editing tiles

Sprites in the first two pages are (generally) used as tiles which make up the terrain. The first page (page 0) contains the basic tiles, while the second (page 1) has alternate textures for those tiles that may also have different properties (the alternate texture is on the same page-relative position as the original tile, so tile 4's alternate texture is tile 68)
Depending on the flags and position, a tile can have various properties in the game.
The flags:
0 - solid tile - entities can stand on it and cannot pass trough. Can be broken.
1 - grabbable - entities with arms can pick and hold this tile from the terrain.
2 - magnet wall - entities with legs can temporarily climb on this tile.
3 - background - this tile serves as background decoration and rendered early (NOTE: tiles that have neither flag 0 or 3 won't be visible!)
4 - bouncy - this tile has extremely high bounciness
5 - explosive - tile will explode when destroyed
6 - damaging - tile will deal contact damage and cannot be stood on
7 - no alt texture - tile will not swap to its page 1 counterpart when its megatile changes texture. Useful for some generic tiles like air or textureless ground, and frees that sprite space for some other sprites.

Some tiles have hardcoded behaviors:
Tile 24 - Always treated as a magnet
Tile 44 - will temporarily swap to 45 when an entity climbs on it
Broken terrain tiles that turn into debris will become tile 15

Tiles 32-39, and 48-55 (bottom left corner, as well as their page 1 variants) have a 5% chance of flipping their lowest bit when placed in a level, for some random variation.

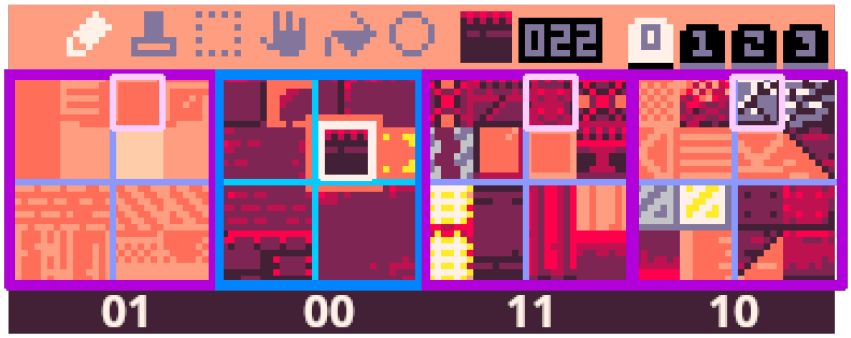
2.3. Editing megatiles

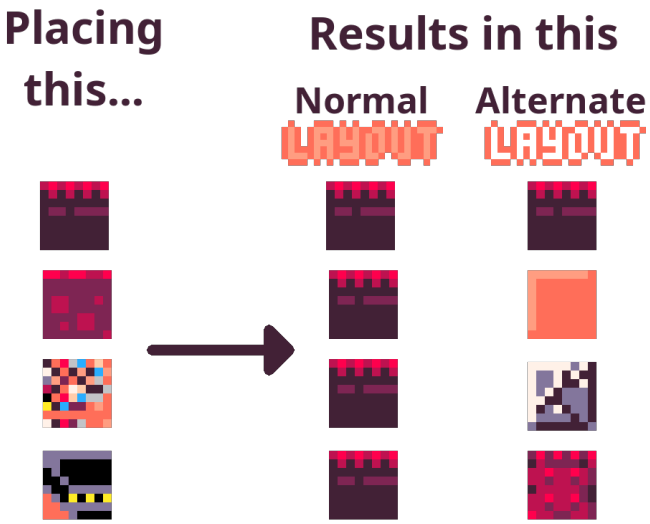
All megatiles are stored in the map from [0,4] to [127,11].
The one at [84,4] is completely unused so it's a good starting point if you want to make a custom megatile. I'd recommend to also load a level first so the global palette is set for easier viewing.

To make a simple megatile, just put the tiles from the first sprite page in its 4x4 area. You can then test in the editor how it looks, and see its alternate texture.
These simple sight-readable megatiles do not have alternate layouts. Making one, however, is a stupidly complex brain-breaking puzzle so brace yourself for the alternate layout algorithm.

The highest two bits of a tile are always ignored in a megatile's default form. Tiles from other sprite pages will simply become their first page variants.
When a megatile enters its alternate layout form, all tiles will flip their bits according to the highest two. If the highest bit is active (original tile is from pages 2 or 3), that tile will flip its fourth bit. The second highest bit being active (original tile is from pages 1 or 3), that tile will flip its third bit.
This basically swaps the tiles into a different 4x4 section of spritespace.

After flipping bits, those tiles will also clear their first and fifth bits, snapping them to the upper left corner of their 2x2 section in the spritespace.





Essentialy, every tile has 3 alternate forms it can take, depending on what page number you give it. Try swapping a tile for its counterpart on a different page and check the results in the level editor.

2.4. Editing the other stuff

As the editor is also fairly large, currently other things would need to be edited manually, either by typing new things into the code or changing the tiles present in the map.

For example, to add a custom entity, you'd need to add a new line inside the "ntt_types" string, format it correctly (the key1,key2/val1,val2 structure explained below), and add 4 new tiles to the mapspace where entities' common properties are stored. (As usual it's likely easier to start by changing or removing existing entities first)

Things stored in string arrays (inside the data section, tab 6 of level editor code):
Level headers
Entities and their properties
Extra level modifications for entities
Entity body types and limb info
Smokes/particle effects
Extra sfx's

Things stored in the map (that would need to be edited manually):
Background images
Common entity info (an array containing each entity's template, mass, radius and sprite)
Links (rope-like connections between entities, used for limbs and terrain attachments)
Guns
Background palettes
Explosions

2.4.1. The string array format

To save on tokens, a lot of the game's data is stored as string-encoded arrays, which are then split with a delimiter (usually ",")

Usually, these strings are either organized as an array (val1,val2,val3,...) where each element modifies a property based on its position, or a pair of key-value arrays (key1,key2,key3,.../val1,val2,val3,...) where the first array contains the names of the properties which will be mofidied, and the second the values.

In the second type, prefixing a value string with "_V_" means it will be converted into the value of a variable with that name (e.g. _V_grav means this will be the value of the variable "grav").

Strings "true","false","nil" and "{}" will also be converted to their constants (true,false,nil and an empty array). Often, true will be shortened to a single letter "t" as it will also evaluate to true in an if statement.

3. Data documentation

This section will try to document most of the game's data and the ways it is stored to make editing it easier.

3.1. The map

[0,0]-[79,3]: Background images
Item size: 8x4
These images are used in the game's backgrounds. 10 can fit in the space, but the settings also allow to use the map data from the palette space next to this (it probably won't look too good in most places, though!)
These can be edited in PICO-8's default map editor.
NOTE: tiles 0 won't be rendered, so backrounds with a lot of empty space will be less CPU expensive to display!

[80,0]-[127,3]: Palettes
Item size: 16x1
Starting direction: down
(This means element #0 will be at 80,0, #1 will be at 80,1, and after reaching the bottom, #4 will be at 96,0)
Stores palettes used by the game. These are 1-indexed, which means the color #0 (the first in PICO-8's sprite viewer) is located in the last spot, while the rest are ordered normally. Currently these would need to be edited manually.

[0,4]-[127,11]: Megatiles
Item size: 4x4
Starting direction: right
All of the megatiles used in the game.

[0,12]-[127,35]: Reserved level space
Item size: any
Technically levels can be stored anywhere, but storing them in a single space keeps things clean and allows levels to share or reuse map data (look in the map viewer at levels 20 & 29 for an example)

[0,28]-[30-ish,43]: Reserved level entity space
Item size: horizontal arrays of 4x1 items
Starting direction: right
Entities present in levels. Can also be present anywhere, and due to this the right borders of this segment are a bit chaotic.

[33,36]-[39,43]: Links
Item size: 7x1
Starting direction: down
The bases for the game's links (ropes, limbs, wires etc.) which connect two entities together. Surrounded by a bit of free space to make orientation slightly easier.

[40,36]-[95,43]: Guns
Item size: 11x1
Starting direction: down
The guns that entities can shoot.

[96,36]-[127,43]: Background layers
Item size: 8x1
All the actual background layers, with image, palette & scrolling data. Saved by the level editor, so these should only be edited manually to copy/paste/quick delete or reorder them.

[0,44]-[127,59]: Sprites
Item size: 8x8
The extra sprites in the shared space. Editable in PICO-8 (as the bottom row of page 2 and the top 3 rows of page 3).

[0,60]-[127,61]: Basic entity info
Item size: 4x1
Starting direction: right
Although most of the entity data is stored in string form, in order to save compressed space, common settings (the "template", radius, mass & sprite) are stored here.

[0,62]-[31,62]: Background palettes
Item size: 4x1
Contains the first 4 colors of a palette used by the backgrounds. The other 12 colors are copied from these 4, meaning one background can have a maximum of 4 colors (usually it's colors 0,1,2,4&5). The final level's background is hardcoded to use default colors instead.

[0,63]-[11+,63]: Explosions
Item size: 3x1
Radius, strength and sound effect of explosions used in the game.

The rest of the map (marked black in the image) is unused. (can possibly fit a level entity array or some custom data)

3.2. Level headers

Below is a list of all the fields of a level's header, along with a description and an example.

3: mayhem square`5`4`170`lly,e_rq/-64,4`0`12`13`10`7`7`1`0`8`7`4096`8

#1: The level's name: If this is the first level of a stage, this is displayed on the splash screen with a background paint (these levels usually have padding in order to show up in the middle and take the entire horizontal space, e.g. " the construction site ")
Otherwise, this is the text that briefly appears in the corner when transitioning to a new level. Leave empty for no text.

#2: The next level to load after completing this one: 1 - indexed, so the first level will have this as "2" to load the second level upon completition.
There are 2 special numbers you can put here:
-1 means this is the last level of a set (completing this displays the results screen).
-2 is the same as -1, but it also makes the level unable to be completed by reaching the right side (it also disables the visual effect). This is usually paired with an entity that will trigger a level transition upon defeat.

#3 & #4: Player spawn coordinates, x & y respectively: Can be edited and previewed in the level editor.

#5: Global variables to set: Upon loading, a level can edit global variables. This is stored in the standard [key1,key2/val1,val2] format. The example level sets the lower level limit to be -64, and the required amount of enemies to clear to 4.

#6 & #7: Level map data coordinates (x & y): Where the upper-leftmost megatile of the level can be found in the map.

#8 & #9: Level size (x & y): How large, in megatiles, is the level.

#10: Music index: Which track will play upon entering the level. If this is the same as the previous level, the track will continue (the layers will still be updated according to the next setting).

#11: Active music channels: This is treated as a bitfield (and displayed as such in the level editor), and determines which music channels will the game enable/disable upon loading the level. Lowest bit targets the leftmost channel (Because of this, the level editor will display the field in reverse order). 0 means inactive, 1 active.

#12: The level's palette index: Ranges from 0 to 11. Palettes are stored in the upper-rightmost corner of the map. The editor will display the memory & map location of the current palette.

#13: The clear color: What color of the palette to use as the clear color/default background.

#14 & 15: Indexes of the level's back, and front backgrounds. Range from 0 to 31.

#16: The memory location of the entity array. This is where all of the level's entities are loaded from, and where the level editor will save entity edits. To convert map coordinates to a memory location, you can use the following calculation:

```
if (y < 32):
        location = x + y*128 + 8192
else:
        location = x + (y-32)*128 + 4096
```

#17: The number of entities in the level. I'd advise you not to edit this directly (unless you're already moving something in the map) as the level editor tracks this automatically when adding/removing entities, and bad stuff can happen if the level tries to read an invalid location.

3.3. Entities

Entity data is divided in 2 parts: the 4 common properties (template entity, mass, radius and sprite) are placed in the map, while all the other properties are inside the "ntt_types" string in the cart. The properties are organized as key/value string, with all the keys listed on one side and all the values being on the other
[e.g. key1,key2,key3/val1,val2,val3]

Templates mean an entity will also inherit all the special properties (keys&values) from that entity. Entity's own properties have higher priority and will overwrite the template's.
Templates are not recursive, and only have a depth of 1.

Below is a description of all the entities present in the game (i'd put this in the comments of a cart but it takes a lot of compressed space).

1. Empty/default entity
2. Player entity, spawned when a level starts.
3. Limb entity. Invisible, small, low mass, high sliperiness (low friction). Used for limbs of all entities.
4. Basic horizontal turret
5. Basic targeting turret. Fires a burst of 3 bullets,
6. Continuous laser turret
7. Small flying drone (the UFO)
8. 1st boss (the big walker tank)
9. Projectile entity. Has contact damage
10. Boss 3's defeat cutscene (part 1)
11. Recovery orb item
12. Temporary tile. the game uses this as a temporary entity when checking collisions. Very high mass
13. Sign. Ignores physics, displays a text box on player collision
14. Boss 3's defeat cutscene (part 2)
15. Secret trinket
16. Grappling hook. Respawns when destroyed
17. Static hazard with continuous downward laser beam
18. Big missle launcher
19. 4th boss (giant airship)
20. Missle projectile (grabbable variant)
21. Sniper targeting recticle (also a projectile)
22. 2nd boss (missle drone)
23. 3rd boss (mirror fight)
24. Enemy template. Not used on its own, but provides common properties for most other enemies
25. Invisible projectile spawner. Spawns downward sawblades by default
26. Giant sawblade projectile
27. Bouncy mushroom
28. Sniper drone
29. Spikeball enemy
30. Alarm antenna. Alerts enemies to the player's position when approached
31. Spawner drone. Spawns humanoid robots by default.
32. Shotgun drone

33. Humanoid robot guard (with a laser sword)
34. Background decal. Used in level 1 for tutorials, but could also be a decoration
35. Fast laser bolt. Invisble by default, but leavs a trail to its parent entity on collision.
36. Missle spider enemy
37. Slow, ungrabbable missle
38. Final boss
39. Laser sword item

3.4. Entity body types

Entities that move use a body type which defines their speed, movement abilities and the amount and type of limbs the entity has (if any).

The string array ntt_b_types defines the body types in the game, and one consists of the following fields, (ordered, with the variable name after conversion wrotten in [square brackets]):

Ground acceleration [g_acc], air acceleration [a_acc], max ground speed [g_max], max air speed [a_max], jumping strength [jump_str]
The next few properties are optional and only needed if the entity has limbs:
Leg length [leg_len] (used for seeking ground, should be lower than actual limb link length), arm length [arm_len] (for grabbing, should also be a bit lower), default standing height [stnd_h], leg movement speed [l_spd], leg movement cooldown [l_cld] (how many frames must pass after a leg moved before the next one can), max target rotation [l_a_r] (how far, in PICO-8 degrees, can the leg look for suitable ground away from its default direction)

Afterwards is N*4 elements, where N is the number of the entity's limbs. Each has:
Limb type (a string, "l" means leg while anything else means arm), standing angle relative to the entity's "down" direction, link type (index of the link in the array), extra link properties in the (key1`key2➡val1`val2) format. Spawned limbs will always be entity #3.

List of all the body types in the array:

1. Standard movement, limbless
2. Humanoid - 2 arms, 2 legs. This is where most of the player's movement stats are
3. Enemy humanoid - very similiar to 2 but has slightly differently colored links and a higher jump
4. Large walker - 2 large legs. Used for boss #1
5. Bipod "spider" - 2 legs in opposite directions, can climb walls.
6. Slow, limbless
7. Fast, limbless
8. Fish, limbless - reduced max speed meaning it can only "fly" in lower gravity (inside liquids)

3.5. Entity extras

This will only be a short description of the modifications, for more precise effects refer to 3.8.

1: nothing. Default behavior
2: proc alert. When a player is in this entity's range, other nearby entities will soon be activated as well.
3: next entity is 11. This entity drops a recovery item upon defeat.
4-7: changes the entity's rope position, if there is one
8: removes an entity's rope and changes its body type to a 2-legged climber
9: sets the gun to 29 (shotgune drone spawner), and disables an entity's "boss" flag
10: enables the entity's "boss" flag
11: adds a rope to the entity
12: when entity is broken, load next level after a short delay
13-15: unused
16: more rope position changes
17: the text box used in tutorial
18: yet more rope
19 & 20: text boxes used in levels 4 & 2
21 - 23: decorative background decals
24: once active, this entity will chase the player over very large distances
25: like 23 but with more range/behavior mods, refer to 3.8. for property details
26: sets the gun to 29
27: entity won't fully count as an enemy (won't show up in stats or unlock the level exit)
28: sets the entity as a collectible "enemy" and "boss" (used with a pickup, on collection counts as an enemy defeat and stops the level music)
29: sets gun to 15
30: entity drops sword on defeat
31: 23 & 26 combined

NOTE: Entity extras are written in the table "ntt_extrainfos_pre", a multiline string separated by newlines in tab 5. However, because some signs use p8scii control codes it needs to be converted into a far less readable single-line string before being exported to the main cart. To do this, select the "compress extras" option in the level editor's main menu in the pause screen.

3.6. Guns

When firing a gun, the entity will spawn a child (or a global) entity with a certain speed and angle, as well as optionally modifying some of the child's, or it's own's properties according to the ones listed in the "prop_mods" list.

Guns are stored in the map, and consist of 11 bytes:

1. Cooldown: when loaded, the entity will have to wait this long before firing the gun. If the entity is an enemy and encounters the player, the cooldown is set to half of this value.

2. Projectile entity: which entity will be spawned upon firing this gun.

3. Speed: The projectile will be fired with this speed, divided by 8. (a byte of 1 means a speed of 0.125)

4. Sfx: The gun will activate this sfx, minus 128 (to enable negatives). sfx 0 (byte 128) means no sound.

5. Angle - in which direction, relative to this entity's looking angle, will this projectile be fired (stored as angle*128+128, meaning a byte of 128 means 0 angle offset, while 129 is 1/128 PICO-8 degrees clockwise of a right-facing entity's looking direction)

6. Is global + burst amount: The lower 7 bits of these indicate that this gun will fire several same-typed projectiles in quick succession. 0 or 1 mean only 1 projectile will be fired. The highest bit, if set, will make the projectile entity global rather than a child of the firing entity (useful for spawned minions or projectiles that can be grabbed and deflected)

7. Burst delay: The amount of frames between the projectiles in a burst. Minimum is 1 frame. (0 gets treated as 1 here as well)

8: Burst angle shift: Each next projectile in the burst will add this value (stored as an angle, so byte is angle*128 +128) to the entity's firing angle. Reset when the burst ends.

9: Next gun to load: After this gun is fired, the entity will load this gun from the array. This allows for one-off guns, as well as more complex gun sequences.

10: Projectile modifications (index): The projectile entity will receive modifications from this index in the prop_mods table.

11: Entity modifications (index): The firing entity will receive modifications from this index in the prop_mods table.

The list of guns:

1. Basic, single-fire
2. Laser sweep
3. Small, ungrabbable missle
4. Player's sword, fast upward swing
5. Player's sword, long downward swing
6. Boss 1's first gun, 4x projectile burst
7. Boss 1 gun 2, 2 missles
8. Boss 1 gun 3, laser ground sweep with delayed explosion
9. 3-projectile burst
10. Grabbable missle
11. Boss 2 gun 1, 3 grabbable missles
12. Boss 2 gun 2, drone summon

13. Boss 2 gun 3, giant downward laser
14. Laser snipe, the projectile duration is the same as the gun cooldown
15. Spawn 3 robot guards slowly, then switch to gun 10
16. Downward big sawblade
17. Drone spawner, has a very large burst delay so spawns 2 after initial delay
18. Empty gun, does nothing
19. Shotgun
20. Constant downward laser beam
21. Boss 3 gun 1 (grappling hook throw)
22. Boss 3 gun 2 (laser snipe), starts with this one
23. Boss 3 gun 3 (small projectile, approach & kick)
24. Boss 3 gun 4 (small projectile, retreat & grab)
25. Robot guard sword (slash up, then 26 and approach player)
26. Robot guard sword (slash down, then 25 and retrat from player)
27. Boss 5 gun 1 (obstacle laser drone spawn)
28. Boss 5 gun 2 (missles)
29. Spawn 5? shotgun drones, then switch to 10 (Boss 5, gun in the last level of world 5 and in world 6)
30.,31.,32. Unused
33. Boss 6 gun 1 (grabbable missles)
34. Boss 6 gun 2 (drones)
35. Boss 6 gun 3 (laser sweep + drop)
36. Boss 6 gun 4 (missles + rise)
37. Boss 6 gun 5 (omnidirectional burst)
38,39,40. Unused

3.7. Other (links, smokes, explosions, sfx's)

Links
Properties: length, [len] strength [str] (link breaks if the two objects' distances are greater than this, 0 means unbreakable), draw type [d_t] (0 & 1 are deprecated, 2 is regular joint, 3 is leg joint (draws a line before the main link to act as a humanoid's torso, 4 is joint that doesn't flip when the entity changes direction)), color [col], width [width], draw order [d_o] (0-4, or else for none) and outline color [outl] (0 is no outline)
The links are:
1. Standard machine connection
2. Mushroom stem
3. Humanoid limb - arm
4. Humanoid limb - leg
5. Enemy limb for spiders & walkers
Some links, like the grapple rope, have dynamic properties & are generated at runtime.

Smokes
Properties: color, radius, sfx (can be negative, 0 for none), [optional] decay rate, [optional] duration
1. Standard entity break

2. Hp/item pickup,
3. Projectile disappear smoke,
4. Boss explosion
5. Laser smoke

Explosions
Properties: radius, strength (times 2, byte 1 means a strength of 0.5. Strength scales kind of quadratically so be careful not to set it too high, around 7.5 is balanced), sfx
1. standard
2. big (grabbable missles)
3. small (delayed laser sweeps, boss 1 & 6)
4. very strong, concentrated (sniper laser)

Sfx's
Alongside regular sfx's, the game also stores a few effects in print statements inside the ex_sfx array. Some sfx properties can access these with negative values (-1 would mean the first of the extra sfx's). The important thing when making these is to make sure that they only overwrite the slot 63 by starting the print statement with "\a63" (This also means only 1 of these extra sfx's can play at once). Currently, only sfx 18 is unused (8 and 63 are intentionally empty, 8 to mess with a track's timings and 63 to store generated sfx's)

Music
Because the current level determines what music channels are active, you can make a track dynamically change as the stage progresses, or potentially store multiple music tracks in one sequence. Sfx 63 signals that a channel will not be activated in that music segment, regardless of a level's settings (useful for making quieter sections of a track)
Segment 0 is used as the menu theme, and segment 6 as ending theme.

Palette
0 - default clear color, background
1 - background
2 - background
3 - special interactables
4 - background deco
5 - background deco
6 - black or very dark
7 - white or very light
8-11 - terrain
12 - player color
13 - metal, interactables
14 - alternate terrain, by default same as 9 but can be different to give tiles a slightly different texture
15 - enemy/electricity color

Text boxes
Used by the "txtb" property, an entity with the "Usgn" update function may display a text box. A text box is specified in a string array, separated by "⬇" and with these properties:
- [string] the string which will be printed inside the box

- [bool] whether the string will be displayed relative to the screen ("true", unlike most bool needs to be exactly this), or in world-space (any other string)
- [num] x coordinates
- [num] y coords
- [num] x size of the box
- [num] y size
- [num] (optional) color of the box
- [num] (optional) color of the box's decorative borders
-- the following properties can be specified, but shouldn't in most cases as they're used for one-off text boxes (like the enemy damage report or the level titles)
- [num] how many frames the box lasts
- [num] the box will begin moving after this many frames
- [num] x velocity
- [num] y velocity

3.8. Useful entity properties & functions

When making custom entities, these are the properties you can set to modify its behaviors. Properties not mentioned here, like pos (position) or vel (velocity) likely won't have good (or any) results upon setting them. As usual mind the case.

Uf - [func] - update function - the entity runs this function every frame
Df - [func] - draw function - also gets run every frame, but in the draw queue depending on the entity's draw order

funcC - [func] - collision function - gets run whenever the entity collides with something
b_f - [func] - break function - gets run when entity is destroyed.

dur - [num] - if present, the entity will disappear after this many frames

rspw - [bool] - if true, the entity respawns at its original position when destroyed. NOTE: only entities directly spawned by the level should have this! If an entity spawns another with this property and that entity attempts to respawn, the game will crash!

expl - [num] - the entity's explosion index. Explosions activate when the entity is destroyed
dly_expl - [num] - explosion, but delayed by 30 frames. Only used if an entity has the "DlEx" function.

lzr_thck - [num] - if the entity has the Blzr break function, this is the thickness of the laser.

Physics ---

Btyp - [num] - index to the entity's body type in the array.

slip - [num] - slipperiness. Parallel momentum is multiplied by this on collision.
bnce - [num] - bounciness. Opposite momentum is mutliplied by this on collision.
Setting any of these 2 higher than 1 could make the physics spin out of control!

grav - [num] - gravity. If unset, this entity will use the level's gravity instead.

ray_iters - [num] - how many iterations will the entity's limb terrain detection use. Higher will find terrain and entities to stand on faster, but will take more processing cycles. Default is 3, the player uses 6.
prst - [bool] - permanent stickyness. Entity can climb walls and attach to magnets without expending magnet charge.

nophys - [bool] - entity is unaffected by physics.
ignS - [bool] - ignores secondaries - this entity will not collide with child entities of other entities (limbs & non-global projectiles)

mass, rds, sprite [num] - already set in the mapspace, but you can modify them to a finer degree in a string

rope - [num] - index of the entity's rope (in the links array)

rX - [num] - the starting point of the rope, relative to the enemy (x offset)
rY - [num] - y offset
rope_e - [string array] - extra modifications to the rope. Format is [key1`key2➡val1`val2]

Combat ---

stmn - [num] - entity's stamina (hp). Also sets its max stamina when spawned
stmnh - [num] - "hard damage". Reduces the entity's max stamina by this amount, until a recovery item is picked up.

iarm - [num] - impact armor. Impacts weaker than this won't deal any damage to the entity. (default is 0)
irss - [num] - impact resistance. Divide impact damage by this. (default 1)

gi - [num] - index of the entity's gun.

dmg - [num] - the entity's contact damage.
kb - [num] - the entity's contact knockback. dmg must be a value (even if 0) for knockback to activate.

drp - [bool] - if the entity can drop kick others by jumping while standing on them.

Enemy behavior ---

ai_p - [func] - passive ai. Entities with update function Uenm (enemies) run this function when not stunned.
ai_a - [func] - active ai. Enemies run this when active.

enemy - [bool*] - whether this entity will count as an enemy in the game's score, and will its health be displayed upon taking damage
NOTE: Unlike the other bools, has three possible behaviors depending on the value ["true" - fully counts as an enemy towards the clear conditions, "false" or "nil" - is not an enemy, (any other string) - doesn't count as an enemy but will still have effects (health report and kick effect) like one, useful for minions]

boss - [bool] - if true, upon defeat, the level's music will stop.

actF - [num] - the enemy will deactivate when player is further than this.
actN - [num] - the enemy will activate when the player closer than this.
rngf - [num] - the enemy will approach when player is further than this.
rngn - [num] - the enemy will retreat when player is closer than this.

p_a - [bool] - proc alert. If enemy is active, will attempt to activate all other enemies, as if the player has entered their actN range.

dash - [num] - every 20 frames, an enemy may randomly choose to jump. This is the chance (0-1) that it will do it.
jumping_d - [num] - if the player is closer than this, the enemy will jump.
j_cldwn - [num] - the entity's jump cooldown

fly - [bool] - if true, the entity can move upwards and downwards while mid-air.
hz - [bool] - horizontal - entity won't move, or shoot up or down.

rck - [bool] - reckless. If true, the enemy will not dodge walls and won't stop firing when next to a wall.

idir - [vector] - entity's input direction. Automatically reset by Uenm function. Can be set to the variables ["v2l","v2r","v2u","v2d"] for left,right,up and down (remember to prefix with "_V_")
b4 - [bool] - Whether the entity has button 4 (jump) pressed. Automatically reset by Uenm function.
b5 - [bool] - Whether the entity has button 5 (grab) pressed. Not managed by aby function.

Drawing & Deco ---

col - [num] - entity's color. Only used when drawing leg joints
outl - [num] - draws an outline around the entity in this color (0 means no outline)

smok - [num] - the index of the entity's smoke. When destroyed, plays this particle effect.

sprW - [num] - for multi-sprite entities. How many sprites is it big horizontally.
sprH - [num] - vertically.
spr_size - [num] - The size of one sprite, in pixels. 8 means default rendering, 16 is double size etc.

f_c - [num] - for animated entities. The number of the entity's frames, to the right of the entity's sprite.
f_l - [num] - how many in-game frames one sprite frame lasts.

d_o - [num] - draw order. Higher means the entity is drawn later (in front of others). Valid is 0-4.

txt - [string] - for items (entities with Citm collision function) - text that is displayed when collecting
txtb - [string] - textbox - displays a text box according to the format specified above (3.7.) when player is inside of this entity's hitbox (this entity needs to ignore physics)
decal - [string] - a print statement that is displayed at the entity's position. Entity needs Ddcl as its draw function.

Other ---

g_i - [bool] - grab immunity. Entity cannot be grabbed.
d_i - [bool] - dropkick immunity. Self explanatory.

tmp_tile - [bool] - tag used for temporary tiles in collision calculations with terrain. This entity cannot be pulled by links.
tile - [bool] - entity will attempt to settle itself when static (convert itself to a tile in the map)

item - [num] - when collected, the entity will apply effects on the player depending on this number. (1-stamina recovery, 2-trinket, else - blade)

Exclusive to gun mods:
ghz - [bool] - projectile will be fired horizontally
gmelee - [bool] - projectile's angle directions won't flip when the parent entity is facing left

+ any entity bodytype properties (max speed, jump strength etc) can only be modified after spawn, making them exclusive to this

These are some useful functions that can be set in the properties that accept one. Note that all of them need to be prefixed with a "_V_" so the system knows to reference it.

e - empty function. Use instead of nil when disabling function properties

Update functions ---
Uply - player update. Entity will stand and can control its movements, but won't actually do anything unless it's set as the "player" variable (which means the player's inputs control it), or some of its input fields have been set (idir, b4 or b5). Can fire guns with b5 if "gi" property is not nil.
Uenm - enemy update. Automatically sets its inputs except b5, detects the player, calls ai functions, and fires guns after their cooldown.
Usgn - sign update. Displays this entity's textbox (txtb) if player is inside its hitbox
move_hmn - move humanoid. Entity will try to passively stand.
move_ctrl - move control. Entity will respond to its input fields.
Umsl - missle update. Moves towards player, unless it is thrown by player.

Draw functions ---
Dntt - default draw entity. Draws its sprite at its position.
Dply - draw for player and humanoid entities. Ofsetted sprite draw. Player also gets expressions.
Ddcl - draws decals by printing this entity's "decal" property.

Collision/break functions ---
Citm - item - if collided with the player, removes this entity and applies effects depending on the entity's "item" property
Chook - grappling hook - on collision, make a link between the collided entity and this entity's parent or thrower. Set the link as the parent's "grapple", making it controllable

lnxt - load next level
d_ld - load next level after a delay
Blzr - draw a laser between this entity's position and its parent for the next few frames.
DlEx - after 30 frames, makes an explosion according to the entity's dly_expl, and also calls Blzr.
rme - removes this entity.

Passive ais ---
funcas - stabilise - entity will try to stand in place
funcaf - fly . - entity will try to stand in place mid-air

Active ais ---
funcaa - follow - move closer to the player if further than far range (rngf), and away if closer than near range (rngn)
funcah - hover - like follow, but move upwards if not higher than the player by this entity's near range [rngn]

3.9. Hardcoded stuff

Some strings, like the level descriptions, are hardcoded (the data is defined directly before it needs to be used) and would need to be edited in the main cartridge's code.

Here are some more notable locations:
Tab line 15 - cartdata name set
Tab 0 line 89 - all level menu descriptions and the text box showing them
Tab 0 line 132 - level splash texts
Tab 0 line 208 - default values for a level.
Tab 0 line 269 - level completition text.
Tab 0 line 298- 307 - game completition conditions, effects and text.
Tab 2 line 18 (global 720) - default entity values.
Tab 7 line 81 (global 1314) - default values for rubble blocks.
Tab 9 line 5 (global 2048) - level names in the menu.